

A Multilingual Groundedness Detector That Runs on a Raspberry Pi: Trained Token Classification Against LLM Judges and English-Only Detectors in 31 Languages

Aghasalim Mustafazada¹ · Teymur Eyvazov²

¹ Co-CEO and Head of AI, ² Co-CEO and Head of Operations; SIBA, "Süni İntellekt Biznes Avtomatlaşdırma" MMC, Baku, Azerbaijan · September 2026 · v4

Abstract. Retrieval-augmented assistants fail their users most often by stating a price, hour, address or phone number that their documents do not support. The open-weights detectors for this failure, Vectara HHEM-2.1-Open and LettuceDetect, are trained on English. We extend our planted-error benchmark to 31 languages across nine scripts (434 answers, 372 with one planted error, 62 grounded controls; identical errors in every language) and add a third kind of detector: a multilingual token classifier of our own, fine-tuned from XLM-RoBERTa-base on 6,647 answers (2,844 synthetic business answers in 30 languages whose unsupported spans were planted and marked by open LLMs, plus RAGTruth), with the decision threshold chosen on held-out data. It catches 352 of 372 planted errors (95 %) with 32 false alarms in 62 grounded answers, in 34 ms per answer on a laptop CPU, and it serves the public demo at grounded.siba.az from a Raspberry Pi 5. On the same cases HHEM-2.1-Open reaches 71 % recall with 22 of 62 grounded answers flagged: its score collapses with script distance from English. In 6 languages it flags every grounded answer, so its recall there measures the script, not the claim. LettuceDetect catches 51 %. The open LLM judges remain the accuracy ceiling: openai/gpt-oss-120b catches 163/163 with 0 false alarms on the 14 languages its daily quota allowed, at 0.70 s per network call, ~21× the detector's latency and a per-token price. Weights (MIT, huggingface.co/aghasalim/grounded-multilingual-base), data, training and evaluation code and every raw output: github.com/aghasalim/groundedness.

Author contributions. A.M. designed the benchmark, the data generation and the model, ran the experiments and wrote the paper. T.E. contributed the operational setting, the business requirements and review.

1. Problem

A business assistant answering from its owner's documents has one failure that costs money: it says something the documents do not say. The detectors built for this in retrieval-augmented generation are trained and evaluated on English: HHEM-2.1-Open, a cross-encoder for factual consistency; LettuceDetect, a ModernBERT token classifier on RAGTruth; Lynx, a Llama-3 fine-tune under a non-commercial licence. A general LLM used as a judge reads every language but costs a network call per answer, has a per-token price, and cannot run inside a small deployment. Earlier versions of this report (v1, v2) established the first two facts on eleven languages. This version asks the practical question: can a small trained model, cheap enough for a Raspberry Pi, do the job outside English?

2. Benchmark

The v2 benchmark is kept unchanged and extended. For each language and two domains (a dental clinic, an electronics shop) a facts block and a grounded answer are templates over five slots (street number, price, opening hours, phone number, return window) plus two extra sentences. The runner plants exactly one error per variant by substituting a slot value (30 → 25 AZN; 9:00–19:00 → 9:00–17:00; No. 12 → 14; one phone digit; 14 → 30 days) or appending a sentence (an invented Sunday opening; an added claim). Every language therefore receives the same seven errors and the same grounded control; 31 languages × 2 domains × 7 answers = 434 cases, 372 with a planted error and 62 grounded. The first eleven language sets (English, Azerbaijani, Russian, Turkish, Ukrainian, Kazakh, Arabic, Persian, Hindi, Indonesian, Vietnamese) were written by hand; the further twenty (German, French, Spanish, Italian, Portuguese, Polish, Dutch, Chinese, Japanese, Korean, Uzbek, Georgian, Hebrew, Bengali, Thai, Swahili, Romanian, Czech, Greek, Hungarian) were translated from the English templates by openai/gpt-oss-120b, validated automatically for slot integrity, and are marked as machine-translated in the case file; native review is invited. A planted error counts as caught only when a detector's named span or claim overlaps the planted text; a score-only detector is credited when it flags the answer.

3. The detector

Data. No multilingual span-labelled corpus of unsupported claims exists, so we generate one. A prompt asks an open LLM (openai/gpt-oss-120b, gpt-oss-20b, qwen/qwen3.8-27b on Groq; gemini-3.8/3.5/2.5-flash for 422 of the documents) for one business document in one of 100 domains (clinics, hotels, tariffs, warranties, timetables, job postings, recipes, changelogs...) in one of 30 languages, a customer question, and four answers: one fully supported, three each with one to three unsupported claims (a changed number, an invented feature, an inverted rule, a wrong day, an overstated promise, an added detail), wrapped in double-bracket markers by the generating model itself. The markers yield exact character spans without a second labelling pass; documents whose markers are malformed, missing from the error answers or present in the clean answer are rejected (711 documents kept, 2,844 answers). Three quarters of the documents are in the eleven benchmark languages, the rest in the further nineteen; the benchmark templates were never shown to the generator. RAGTruth's human-labelled English responses (4,063 answers, hallucinated ones kept in full and clean ones matched in number) are added so the model also sees real model output on long passages.

Model. XLM-RoBERTa-base (278M parameters, 100 languages) with a two-way token head. The input is the answer followed by the sources, 384 tokens, the sources truncated first; only answer tokens carry labels (1 inside an unsupported span). Cross-entropy with the positive class weighted 5.0 $\times$, AdamW, learning rate 3e-05, batch 8, 3 epochs on an Apple M4 GPU in under three hours. After training the decision threshold is chosen on a held-out set of documents to maximise answer-level F1 (threshold 0.5; answer-level F1 on the held-out dev set (P=0.85, R=0.92)). At inference a span is a maximal run of answer tokens above threshold; the score is one minus the share of the answer's characters inside spans. Served with dynamic int8 quantisation the model answers in about 34 ms on a laptop core and under a second on a Raspberry Pi 5.

4. Results

Detector	Kind	Languages	Errors caught	False alarms	Median latency
openai/gpt-oss-120b	LLM judge (Groq)	14	163/163 (100 %)	0/28	0.70 s
gemini-3.8-flash	LLM judge (Google)	11	132/132 (100 %)	0/22	2.88 s
qwen/qwen3.8-27b	LLM judge (Groq)	21	214/215 (100 %)	1/36	0.34 s
openai/gpt-oss-20b	LLM judge (Groq)	17	200/203 (99 %)	2/34	0.44 s
gemini-3.5-flash	LLM judge (Google)	11	129/132 (98 %)	0/22	5.05 s
gemini-2.5-flash	LLM judge (Google)	11	129/132 (98 %)	0/22	5.54 s
grounded-multilingual-base (ours)	trained classifier (ours)	31	352/372 (95 %)	32/62	0.03 s
HHEM-2.1-Open	trained classifier	31	265/372 (71 %)	22/62	0.04 s
LettuceDetect-base (en)	trained classifier	31	191/372 (51 %)	28/62	0.05 s
allam-2-7b	LLM judge (Groq)	11	67/132 (51 %)	21/22	0.32 s

Table 1. Planted errors caught and grounded answers wrongly flagged, all cases each detector was run on. The Google judges were run on the first eleven languages only (credits exhausted); every other row covers all 31. Latency for the judges is per network call from Baku; for the classifiers, per inference on a laptop CPU.

Detector	price	hours	address	phone	days	sunday	extra
openai/gpt-oss-120b	28/28	14/14	27/27	28/28	13/13	27/27	26/26
gemini-3.8-flash	22/22	11/11	22/22	22/22	11/11	22/22	22/22
qwen/qwen3.8-27b	36/36	19/19	35/36	36/36	17/17	35/35	36/36
openai/gpt-oss-20b	34/34	17/17	32/34	34/34	16/17	34/34	33/33
gemini-3.5-flash	22/22	8/11	22/22	22/22	11/11	22/22	22/22
gemini-2.5-flash	22/22	8/11	22/22	22/22	11/11	22/22	22/22
grounded-multilingual-base (ours)	62/62	31/31	58/62	53/62	28/31	59/62	61/62
HHEM-2.1-Open	62/62	14/31	40/62	23/62	15/31	54/62	57/62
LettuceDetect-base (en)	29/62	24/31	26/62	19/62	1/31	40/62	52/62
allam-2-7b	18/22	3/11	9/22	10/22	6/11	8/22	13/22

Table 2. Recall by error type, all languages. The first five are slot substitutions; "sunday" and "extra" are appended sentences.

Planted errors caught, out of 12 per language

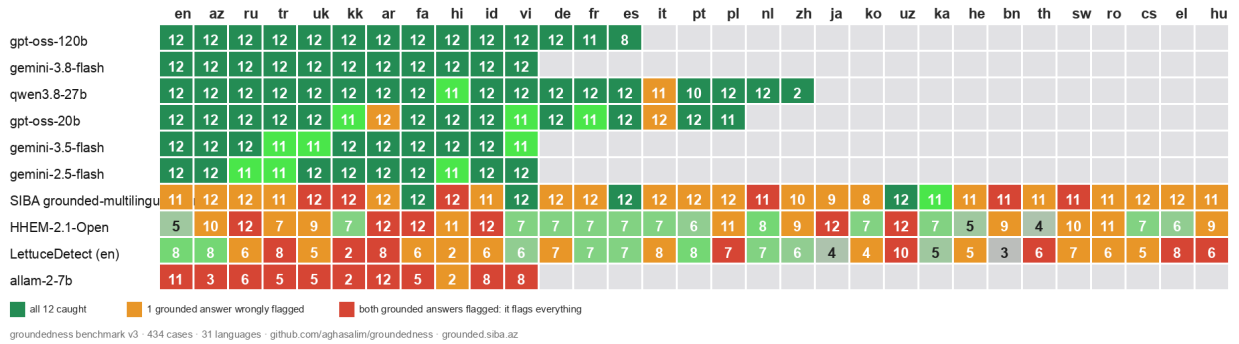


Figure 1. Planted errors caught out of 12 per language. Green: all twelve. Amber: one of the two grounded answers wrongly flagged; red: both, meaning the detector flags everything in that language. Grey: not run.

Our detector. 352/372 planted errors caught with 32/62 false alarms; all twelve caught in 17 languages and no false alarm in 5 of 31. The weakest languages are Korean (8/12), Japanese (9/12), Chinese (10/12). By error type (Table 2) the slot substitutions inside a sentence are the harder cases for a token classifier than the appended sentences, the opposite of the judges, which occasionally under-quote an appended claim. The false alarms have one shape: a grounded answer restates a fact in a different surface form, as in "Monday to Saturday" for the source's "Mon-Sat", "Nizami 12" for "Nizami küç. 12", and the classifier, trained on sources that state each fact once in one form, marks the paraphrase. That is a data fix (paraphrased sources in the generator), not a modelling limit, and it is the first item of future work. This is the row that did not exist before: a detector under 300 MB, no API, that reads Kazakh, Persian and Vietnamese.

The judges. The Groq free tier's daily token budget, spent largely on data generation, let the judges cover only part of the extended set: openai/gpt-oss-120b 163/163 caught, 0 false alarms on 14 languages; qwen/qwen3.8-27b 214/215 caught, 1 false alarms on 21 languages; openai/gpt-oss-20b 200/203 caught, 2 false alarms on 17 languages. Within their coverage they remain near-perfect in every script, as in v2. Accuracy on this benchmark is essentially solved by a prompt. The cost is latency, a network dependency and a per-token bill, which is what the trained model removes; the remaining judge rows will be completed as quota allows and the repository tables are the live record.

HHEM-2.1-Open. 265/372 caught reads as 71 % recall, but the false-alarm column explains it: 22/62 grounded answers flagged, both of them in 6 languages (Russian, Arabic, Persian, Indonesian, Japanese, Uzbek). Its consistency score falls with script distance from English regardless of content: the Latin-script European languages keep it near its English behaviour, the Cyrillic, Arabic, CJK and Indic ones do not; in English it catches 5 of 12 with no false alarm. Outside its training script it detects the language, not the claim. **LettuceDetect** catches 191/372 with 28/62 false alarms, recall falling with script distance from its English training data.

5. Limitations

The benchmark is formulaic by construction (short answers, numeric slots, one error each) and measures whether a detector reads the language and localises a claim, not whether it catches paraphrase, implication or omission; those are the next case set and will lower every row. Twenty of the 31 language sets are machine translations, checked for slot integrity but not yet by native speakers. The training data is synthetic: the generating models' own errors and marker placement are the labels, and their style is one register (a business answering a customer); the RAGTruth share is English only. Our detector marks spans but does not rewrite the answer, so a deployment that wants the corrected text still needs a generator for the flagged cases. Threshold calibration used our own held-out documents, not the benchmark. The English classifiers were run with their default thresholds; tuning HHEM per language would raise its English recall and would not repair its behaviour on other languages, whose scores collapse regardless of content.

6. Availability

Weights and the serving code, the data generator, the training and calibration scripts, the case templates for all languages, both runners, the raw per-call outputs for every detector, the site and this report's generator are at github.com/aghasalim/groundedness under the MIT licence. The detector answers publicly and without a key at grounded.siba.az (POST /check) and through the package (*pip install groundedness; check(answer, sources, model="siba")*). The same judge runs on every answer of SIBA's customer assistant across its channels.

Reproduce.

```
python train/gen_data.py --calls 4000 && python train/prepare.py
python train/train.py --epochs 3 --bs 8 --pos-weight 5 --out train/out/run2
python train/eval_bench.py train/out/run2 && bash benchmark/run_v3.sh
```

References

- Vectara. HHEM-2.1-Open and the Hallucination Leaderboard. huggingface.co/vectara/hallucination_evaluation_model.
- Kovács, Á. et al. LettuceDetect: A Hallucination Detection Framework for RAG Applications. arXiv:2502.17125, 2025.
- Ravi, S. et al. Lynx: An Open Source Hallucination Evaluation Model. arXiv:2407.08488, 2024.
- Niu, C. et al. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. ACL 2024.
- Conneau, A. et al. Unsupervised Cross-lingual Representation Learning at Scale (XLM-R). ACL 2020.
- Mustafazada, A. Do English Hallucination Detectors Work in Other Languages? Technical report v2, SIBA, 2026. github.com/aghasalim/groundedness.
- Aghayev, F., Ahmadbayli, E. SIMURG: Zero-Leak Online Detection of LLM Decoding Corruption in Production Streams. HAL-X AI, 2026.